

Практика 1. Модель та навчання в 1D бінарному дискретному середовищі

Курс: Вступ до загального штучного інтелекту

Осауленко Вячеслав

2026-01-01

Зміст

1	Теоретичне підґрунтя	2
1.1	Середовище	2
1.1.1	Топологія	2
1.2	Агент	2
1.3	Сприйняття	2
1.4	Модель світу та задача передбачення	3
1.5	Два підходи до моделі передбачення	3
1.5.1	Підхід 1 — Словниковий (Егоцентричний)	3
1.5.2	Підхід 2 — Симуляційний (Аллоцентричний)	4
2	Структура коду	4
2.1	Клас World	4
2.2	Клас Agent	5
2.2.1	Ключові атрибути	5
2.2.2	Метод perceive()	5
2.2.3	Метод predict() — реалізовано (словниковий підхід)	5
2.2.4	Метод update_model() — реалізовано (словниковий підхід)	5
2.2.5	Метод execute_action()	6
2.3	Функція main()	6
3	Завдання	6
3.1	Завдання 1 — Клонування репозиторію	6
3.2	Завдання 2 — Реалізація симуляційного підходу	6
3.3	Завдання 3 — Порівняльний аналіз	7
3.3.1	3.1. Точність передбачення	7
3.3.2	3.2. Розмір моделі	8
3.4	Завдання 4 — Вимірювання швидкості	9
3.5	Завдання 5 — Визначення кільцевої топології	9
3.6	Завдання 6 — Стискання моделі (відкрите питання)	9
4	Критерії оцінювання	10
5	Корисні матеріали	10

1 Теоретичне підґрунтя

1.1 Середовище

Розглянемо найпростіше можливе просторове середовище — **1D бінарне дискретне**. Воно є рядком із N бітів:

$$W_t = (w_0, w_1, \dots, w_{N-1}), \quad w_i \in \{0, 1\}$$

Час дискретний: $t = 0, 1, 2, \dots$. У базовому варіанті середовище **статичне**: $W_{t+1} = W_t$.

Кількість можливих станів: $|S| = 2^N$. При $N = 50$ це $\approx 10^{15}$ — вже непідйомно для повного перебору.

1.1.1 Топологія

Середовище може мати різну топологію. У коді реалізовано **кільце (ring)**:

$$r_{t+1} = (r_t + a_t) \bmod N$$

Це означає, що агент, дійшовши до правого краю, з'являється зліва — і навпаки. Меж немає.

Інші варіанти, які можна розглядати:

- **Лінійне** — є два кінці; поводження на межі задається окремими правилами (відбиваючі або поглинаючі межі)
- **Нескінченне** — $r \in \mathbb{Z}$, меж немає взагалі
- **Стрічка Мьобіуса** — кінці з'єднані з перекрутом (топологічна цікавинка)

1.2 Агент

Агент характеризується:

- **Позицією** $r_t \in \{0, \dots, N-1\}$
- **Дією** $a_t \in \{-1, 0, +1\}$ — крок вліво, стоп, крок вправо

1.3 Сприйняття

В момент часу t агент отримує **сприйняття** — вікно навколо своєї поточної позиції:

$$o_t \in \{0, 1\}^M, \quad M = 2 \cdot S + 1$$

де S — радіус сприйняття (perception_radius у коді). Агент зчитує M сусідніх бітів:

$$o_t^j = w_{(r_t - S + j) \bmod N}, \quad j = 0, 1, \dots, M-1$$

Центральний елемент (індекс S) — позиція самого агента.

Сприйняття зазвичай **обмежене** ($M \ll N$): агент не бачить усього середовища одразу. Також може бути **зашумленим**:

$$o'_t = o_t \oplus \varepsilon, \quad \varepsilon \sim \text{Bern}(q)$$

де $\oplus$ — XOR, а q — імовірність інверсії окремого біта.

1.4 Модель світу та задача передбачення

Модель світу M_t — внутрішнє представлення агента, яке оновлюється після кожного спостереження. Вона не може зберігати повну інформацію про W (бо $K \ll N$), але повинна бути достатньою для прийняття рішень.

Одна з ключових цілей агента — **передбачення**:

$$R_t = -\|o_{t+1} - \hat{o}_{t+1}\|, \quad \hat{o}_{t+1} = g(M_t, a_t)$$

Агент у момент t , знаючи поточне сприйняття o_t та заплановану дію a_t , має передбачити, яким буде сприйняття o_{t+1} після виконання дії.

1.5 Два підходи до моделі передбачення

Основна відмінність між підходами — **система відліку**, в якій будується модель.

1.5.1 Підхід 1 — Словниковий (Егоцентричний)

Агент запам'ятовує пари (поточне сприйняття, дія) $\rightarrow$ наступне сприйняття:

$$\text{model}[(o_t, a_t)] = o_{t+1}$$

Модель — це таблиця (словник), де відносний патерн навколо агента + дія визначають наступний відносний патерн. Система відліку прив'язана до **самого агента** — звідси назва «егоцентрична».

Властивості:

Характеристика	Значення
Розмір запису	пара $(o_t, a_t) \rightarrow o_{t+1}$
Макс. кількість записів	$2^M \times 3$
Залежить від N ?	Ні
Залежить від M ?	Так, експоненційно
Потребує знання позиції?	Ні

Максимальний розмір словника: $3 \cdot 2^M$ записів. При $S = 3$ маємо $M = 7$, тобто до $3 \cdot 128 = 384$ записів — незалежно від N .

1.5.2 Підхід 2 — Симуляційний (Аллоцентричний)

Агент зберігає **копію всього середовища** $\hat{W}$ і підтримує оцінку своєї поточної позиції $\hat{r}$. Система відліку — **глобальна, незалежна від агента**.

Для передбачення:

1. Обчислити прогнозовану позицію: $\hat{r}_{t+1} = (\hat{r}_t + a_t) \bmod N$
2. Зняти вікно з внутрішньої моделі: $\hat{o}_{t+1} = \hat{W}[\hat{r}_{t+1} - S : \hat{r}_{t+1} + S]$

Для оновлення моделі: якщо отримане сприйняття o_{t+1} не збігається з передбаченим $\hat{o}_{t+1}$, відповідні біти $\hat{W}$ оновлюються.

Властивості:

Характеристика	Значення
Розмір моделі	N бітів
Залежить від N ?	Так, лінійно
Залежить від M ?	Ні (для розміру)
Потребує знання позиції?	Так

2 Структура коду

Файл `practice_1/1_binary_discrete.py` містить дві основні класи та функцію запуску.

2.1 Клас World

```
class World:
    def __init__(self, size=20):
        self.size = size
        self.state = np.random.randint(0, 2, size=size) # випадковий бінарний вектор

    def get_value(self, position):
        return self.state[position % self.size] # кільцева індексація

    def set_value(self, position, value):
        self.state[position % self.size] = value

    def visualize_opencv(self, agent_pos=None, cell_size=40):
        ... # повертає BGR-зображення для OpenCV
```

World — пасивний: зберігає стан, не змінюється з часом (статичне середовище). Кільцева індексація реалізована через `% self.size`.

2.2 Клас Agent

2.2.1 Ключові атрибути

```
class Agent:
    def __init__(self, world, position=0, perception_radius=2):
        self.world = world          # посилання на середовище
        self.position = position    # поточна позиція
        self.perception_radius = perception_radius # радіус сприйняття S
        self.prediction_model = {}  # словник: (o_t, a_t) -> o_{t+1}
        self.last_perception = None # o_{t-1} для оновлення моделі
        self.last_action = None     # a_{t-1} для оновлення моделі
        self.predicted_perception = None # поточний прогноз
```

2.2.2 Метод perceive()

```
def perceive(self):
    perception = []
    for offset in range(-self.perception_radius, self.perception_radius + 1):
        pos = (self.position + offset) % self.world.size
        perception.append(self.world.get_value(pos))
    return np.array(perception)
```

Повертає вікно довжиною $M = 2S + 1$ навколо поточної позиції агента з урахуванням кільцевої топології.

2.2.3 Метод predict() — реалізовано (словниковий підхід)

```
def predict(self, action):
    current_perception = tuple(self.perceive())
    key = (current_perception, action)

    if key in self.prediction_model:
        self.predicted_perception = np.array(self.prediction_model[key])
    else:
        self.predicted_perception = None # немає даних — немає прогнозу

    return self.predicted_perception
```

Пошук у словнику: якщо пара (поточне сприйняття, дія) вже зустрічалась — повертає збережений результат. Інакше — None.

2.2.4 Метод update_model() — реалізовано (словниковий підхід)

```
def update_model(self):
    if self.last_perception is not None and self.last_action is not None:
        current_perception = tuple(self.perceive())
        key = (self.last_perception, self.last_action)
        self.prediction_model[key] = current_perception
```

Після виконання дії та отримання нового сприйняття — додає або оновлює запис у словнику.

2.2.5 Метод `execute_action()`

```
def execute_action(self, action):
    self.last_perception = tuple(self.perceive()) # запам'ятати поточне
    self.last_action = action

    if action == 'left':
        self.move_left()
    elif action == 'right':
        self.move_right()
    elif action == 'stay':
        self.stay()

    self.update_model() # оновити після переміщення
```

Послідовність: зберегти стан → виконати рух → оновити модель новими даними.

2.3 Функція `main()`

Інтерактивна симуляція через OpenCV. Три вікна:

Вікно	Зміст
World State	Повний стан середовища (червоне поле = позиція агента)
Sensory Input	Поточне сприйняття агента (вікно M бітів)
Predicted Input	Прогноз наступного сприйняття

Управління: **a** — вліво, **d** — вправо, **s** — стоп, **v** — зберегти модель, **l** — завантажити, **q/ESC** — вийти.

3 Завдання

3.1 Завдання 1 — Клонування репозиторію

```
git clone https://github.com/hronoses/intro2agi_course
cd intro2agi_course
git checkout -b your_branch_name
```

3.2 Завдання 2 — Реалізація симуляційного підходу

Реалізуйте **другий підхід** передбачення наступного стану середовища.

Ідея: Модель середовища — це копія самого середовища $\hat{W}_t \in \{0, 1\}^N$. Агент також зберігає оцінку своєї **поточної позиції** $\hat{r}_t$. Вважаємо, що механіка дій відома: $\hat{r}_{t+1} = (\hat{r}_t + a_t) \bmod N$.

Що потрібно реалізувати:

Додайте до класу Agent другу «голову» — атрибути і методи симуляційного підходу. Наприклад:

```
# Нові атрибути в __init__:
self.world_model = None          # внутрішня копія середовища (N бітів)
self.model_position = None      # оцінка поточної позиції агента
```

Метод predict_sim(action) — передбачення на основі симуляції:

```
def predict_sim(self, action):
    # 1. Обчислити прогнозовану позицію
    #     next_pos = (self.model_position + delta) % N
    # 2. Зняти вікно з self.world_model навколо next_pos
    # 3. Повернути прогноз
    pass
```

Метод update_model_sim() — оновлення після спостереження:

```
def update_model_sim(self):
    # 1. Оновити model_position на основі виконаної дії
    # 2. Порівняти поточне сприйняття з відповідним вікном у world_model
    # 3. Записати нові спостереження у world_model
    pass
```

Підказка

При ініціалізації world_model можна заповнити нулями або випадковими значеннями. Агент поступово «малює» своє внутрішнє середовище по мірі переміщення.

Чому два підходи фундаментально різні?

Словниковий підхід не вміє узагальнювати: він знає лише ті пари (сприйняття, дія), які вже бачив. Якщо агент потрапляє у нову позицію — прогнозу немає.

Симуляційний підхід після **повного обходу** середовища вміє передбачати для **будь-якої позиції** без нових даних. Але він вимагає пам'яті $O(N)$.

3.3 Завдання 3 — Порівняльний аналіз

Проведіть систематичні експерименти та побудуйте графіки.

3.3.1 3.1. Точність передбачення

Для кожного методу виміряйте **похибку передбачення** як функцію параметрів:

$$\text{error} = \frac{1}{T} \sum_{t=1}^T \|o_t - \hat{o}_t\|_1$$

де T — кількість кроків.

Побудуйте графіки:

- Error vs. N (розмір середовища) при фіксованому S
- Error vs. S (радіус сприйняття) при фіксованому N

Схема вимірювання

```
def run_experiment(world_size, perception_radius, n_steps=1000):
    world = World(size=world_size)
    agent = Agent(world, position=0, perception_radius=perception_radius)

    errors_dict = []
    errors_sim = []

    for _ in range(n_steps):
        action = random.choice(['left', 'right'])

        pred_dict = agent.predict(action)
        pred_sim = agent.predict_sim(action)

        agent.execute_action(action)
        actual = agent.perceive()

        if pred_dict is not None:
            errors_dict.append(np.sum(actual != pred_dict))
        if pred_sim is not None:
            errors_sim.append(np.sum(actual != pred_sim))

    return np.mean(errors_dict), np.mean(errors_sim)
```

3.3.2 3.2. Розмір моделі

Порівняйте кількість **бітів пам'яті**, яку використовує кожна модель після T кроків:

- **Словниковий**: кількість записів $\times$ розмір одного запису ($2M$ бітів на запис)
- **Симуляційний**: N бітів (завжди фіксований)

Побудуйте графіки залежності розміру від N та від M .

Теоретичні межі

Словниковий підхід: максимум $3 \cdot 2^M$ записів — **не залежить** від N .
 Симуляційний підхід: рівно N бітів — **не залежить** від M .
 Яка модель компактніша при великому N і малому M ? А при малому N і

великому M ?

3.4 Завдання 4 — Вимірювання швидкості

Для кожного методу виміряйте час у **мікросекундах**:

- **Швидкість передбачення** (`predict / predict_sim`)
- **Швидкість оновлення моделі** (`update_model / update_model_sim`)

Рекомендується використовувати `timeit` або `time.perf_counter`.

```
import time

start = time.perf_counter()
for _ in range(10_000):
    agent.predict('right')
elapsed = (time.perf_counter() - start) / 10_000 * 1e6
print(f"Dict predict: {elapsed:.3f} мкс")
```

Заповніть таблицю:

Метод	Передбачення (мкс)	Оновлення (мкс)
Словниковий		
Симуляційний		

3.5 Завдання 5 — Визначення кільцевої топології

Подумайте: **як агент може визначити**, що він знаходиться в середовищі з кільцевою (модульною) топологією?

Агент бачить лише $M \ll N$ бітів одночасно. Він не знає N і не бачить країв.

Підказки для роздумів:

- Якщо агент рухається лише вправо — через скільки кроків він «повернеться» на те саме сприйняття?
- Чи може агент відрізнити два випадки: (а) він повернувся на початкову позицію; (б) він зустрів ідентичний патерн в іншому місці?

3.6 Завдання 6 — Стискання моделі (відкрите питання)

Розмір симуляційної моделі лінійно зростає з N . При $N \rightarrow \infty$ зберігати повну копію неможливо.

Поміркуйте над такими питаннями та запишіть свої міркування:

1. **Стиснення патернів.** Якщо середовище структуроване (містить патерни, що повторюються), чи можна зберігати не кожен біт, а правило генерації? Наприклад, клітинний автомат із правилом 110 генерує складний, але детермінований патерн — і зберігається лише правило (8 бітів!), а не весь стан.

2. **Часткова карта.** Чи потрібно агенту зберігати всі N бітів, якщо він відвідав лише $K \ll N$ позицій? Як організувати «ледачу» карту (lazy map), яка зростає в міру дослідження?
3. **Нескінченне середовище.** Якщо $N = \infty$ — яку структуру даних використати? Що буде «одиницею пам'яті» в цьому випадку?
4. **Ймовірнісна модель.** Замість бінарного значення $\hat{w}_i \in \{0, 1\}$ зберігати ймовірність $\hat{p}_i = P(w_i = 1)$. Чи це зменшує розмір моделі? Яку точність передбачення дає такий підхід?

4 Критерії оцінювання

#	Критерій	Бали
2	Реалізація симуляційного підходу (методи <code>predict_sim</code> , <code>update_model_sim</code>)	30
3	Графіки точності та розміру моделі	20
4	Вимірювання швидкості, заповнена таблиця	15
5	Письмова відповідь або реалізація алгоритму виявлення кільця	20
6	Письмові міркування про стискання (мін. 2 пункти)	15
	Разом	100

5 Корисні матеріали

- Базовий код: `practice_1/1_binary_discrete.py`
- Лекційні матеріали: `lectures/lecture_4/presentation.qmd`
- NumPy документація: numpy.org/doc
- timeit модуль: docs.python.org/3/library/timeit